From an Intent Cascade to a Usable Offline Assistant: Request-Kind Routing, Grounded Notes Q&A and a Sandboxed Code Loop on a Laptop CPU

[Author name]
[Affiliation] [email]

Draft, 28 October 2026 (Kestrel v3, tag `v3-frozen`)

Abstract

Kestrel is a personal assistant that runs entirely on a laptop CPU. Its first version routed requests among nine intents with a calibrated cascade: eight app actions, and *unsupported*, which it refused. We describe and evaluate its third version, which adds written answers (chat, email drafts, rewriting, translation), notes found by meaning, questions answered from the user's notes, and small Python programs checked in a sandbox, all offline, on a 6-core laptop with 14.95 GiB of RAM and a single 4-bit small language model (Gemma-4-E2B). A temperature-scaled linear head on a frozen multilingual encoder chooses the request's *kind* before the old cascade runs; when it is unsure, the language model scores the kinds, and when that is unsure too, a skill is proposed to the user as a question (a request that leans to the app takes the old path). On a held-out English–Hinglish test split ($n = 280$) the decision was right for 256/280 requests (91.4%, 95% CI [87.6%, 94.2%]), started a skill nobody asked for 6 times and asked the user 13 times; 256 of those requests had been scored once before, and on the 24 fresh ones the decision was right for 23/24. Retrieval by meaning found the right note for 14/18 test queries that keyword search missed and added no note to queries that had none, once Romanized Hindi queries were first put into English (on development data this raised Hinglish from 1/9 to 9/9). Grounded notes Q&A answered 19/19 answerable test questions correctly and declined all 6/6 unanswerable ones. On 24 held-out code requests the model was right 21/24 times, but only 3/6 on scripts that read input, and its sandbox retry loop never fired. Mapping the model file cut the model's non-reclaimable memory from 3247 to 1144 MiB, and, with a cached prompt prefix as well, the worker became ready in 2.0 s instead of the previous version's 19.1 s. An end-to-end acceptance run passed 34/36 requests. All evaluation sets are small and synthetic; unlike the pre-registered first study, v3's protocol was fixed step by step, and we say where that weakens a result.

Keywords: on-device assistants; small language models; request routing; calibration; retrieval; grounded question answering; code generation; sandboxing; code-mixed Hinglish; privacy.

1 Introduction

A personal assistant that never sends the user's words to a server has to do all of its work on the user's own machine. Kestrel is such an assistant for notes, tasks and reminders. Its first version (v1) studied one question: can a calibrated cascade of keyword rules, a frozen sentence encoder and a 4-bit small language model (SLM) classify a request into one of nine intents (eight app actions and *unsupported*) as accurately as the SLM alone, and faster? That study was pre-registered and reported separately (the v1 paper); its primary hypothesis was supported on CLINC150. A second version made the system usable day to day: task commands in English and Hinglish, desktop reminders, an always-on service that loads the SLM on demand, and a local browser page.

Both versions refused anything outside the eight app actions. "Write an email to my landlord", "translate this into Hindi", "when is my mock interview?" or "fix this function" ended as *unsupported*, or worse, in a wrong intent. Version 3 (v3) set out to answer these on the same laptop, completely offline and at no cost: tasks and reminders; notes search by meaning and summaries; email drafts in English and Hinglish; rewriting and translation; normal chat; questions answered from the user's notes; and, with a compile and test loop, small Python scripts and functions, explanations of code and fixes of simple errors.

This paper reports how v3 does that and how well each part works. Its contributions are:

1. A *kind decision* before the intent cascade: one encoder pass and a temperature-scaled linear head choose between the old app path and six skills; below a threshold the SLM scores the kinds, and below a second threshold a skill is proposed to the user as a question. It needs no extra model and leaves the research cascade unchanged (Section 4).

2. Retrieval by meaning over the user's notes, added after the unchanged keyword search, with Romanized Hindi queries put into English by the SLM first, because the encoder does not handle them (Section 5).

3. Notes Q&A that answers only from the closest notes, checks numbers and names against them, and says when the notes do not answer (Section 5).

4. A code loop in which the same 2B-class SLM writes, fixes and explains small Python programs, checked in a network-less sandbox with at most two retries on the sandbox's own errors (Section 6).

5. Two engineering measurements that make an always-on service practical on 15 GiB of RAM: memory-mapped weights and a cached primed prompt prefix (Section 3).

6. An evaluation on five small English–Hinglish sets with one test run each, an end-to-end acceptance run, and the negative results: notes questions and rewrite requests are often not recognised, scripts that read input fail half the time, and some answers come back in the wrong language.

2 Related Work

Routing and cascades. Model cascades answer with a cheap model when it is confident and defer otherwise (Varshney and Baral, 2022); FrugalGPT (Chen et al., 2024a) and RouteLLM (Ong et al., 2025) route between language models of different cost, and confidence-based deferral can fail under distribution shift (Jitkrittum et al., 2023). Our kind decision is such a cascade over *tasks* rather than models: the encoder head decides, the SLM is consulted only when the head is unsure, and the last resort is a question to the user. Temperature scaling (Guo et al., 2017) and the selective-prediction view (Geifman and El-Yaniv, 2017), both used in v1, carry over.

Encoders, intents and code-mixing. We reuse v1's frozen multilingual E5 encoder (Wang et al., 2024), a sentence-embedding model in the line of Sentence-BERT (Reimers and Gurevych, 2019). Intent detection with out-of-scope requests (Larson et al., 2019) and an encoder that defers to an LLM on uncertainty (Arora et al., 2024) are the closest classification settings; code-switched task-oriented parsing has been studied for Spanish–English (Einolghozati et al., 2021). We found that the encoder does not place Romanized Hindi queries near English notes with the same meaning, and route them through translation instead.

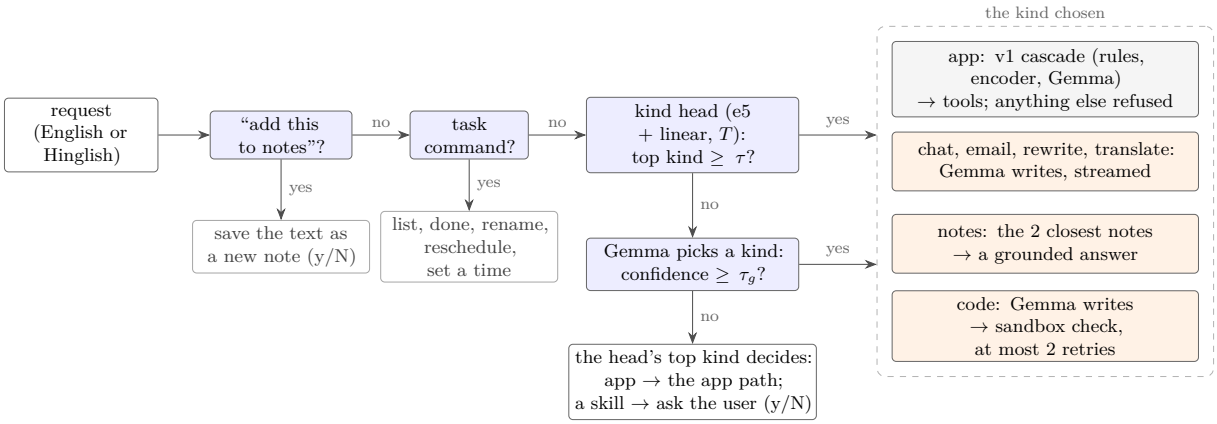


Figure 1: How v3 handles a request. Saving a note (read from the text as typed) and task commands are checked first. If the kind head's top kind reaches τ, it decides: *app* takes the unchanged app path (v1's cascade and tools), a skill is written. Below τ, whatever the top kind, Gemma's choice decides if its confidence reaches τ_g; otherwise the head's top kind decides, except that a skill is proposed to the user as a y/N question.

Retrieval and grounded answers. Retrieval-augmented generation conditions a generator on retrieved passages (Lewis et al., 2020). SQuAD 2.0 added questions that the passage does not answer, so that a reader must also abstain (Rajpurkar et al., 2018); our notes Q&A set has such questions, and the system must answer "not in your notes" for them.

Code generation and repair. HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) judge generated Python by running it against unit tests; we judge our requests by tests the model never sees, on far fewer items, with English and Hinglish phrasings and a fourth type (explanations, judged by required facts). Self-debugging (Chen et al., 2024b) feeds execution results back to the model; our loop does this only for the sandbox's own errors, after the model's own tests proved unreliable (Section 8.4).

The models. The SLM is Gemma-4-E2B (Gemma Team, 2026) at 4-bit, run with llama.cpp (ggml-org, 2026); two Qwen3 models (Yang et al., 2025) were compared for code on development data.

3 System

Figure 1 shows the path of a request. Everything runs in one always-on process, `kestrel serve`, that keeps the rules and the encoder (multilingual-e5-small, 8-bit) loaded and starts the SLM in a worker process on the first request that needs it, stopping it after ten idle minutes. The terminal client and the browser page talk to it over a local socket and a localhost HTTP server; answers from the SLM are streamed as they are written, except notes answers and note summaries, which are shown after a grounding check (Section 5). The research code paths, the nine-intent ontology, its datasets and its frozen configurations are untouched: v3 adds an application layer in front of them.

Skills. *Chat* answers general questions and explanations; every chat answer ends with a visible note that an offline small model wrote it and may be wrong. Current facts, live data and calculations stay on the app path, where they are refused. *Email* writes a draft (sending stays an app request). *Rewrite* changes a given text in its own language, and *translate* puts it into

another; a short note after the request names the output language and script ("hindi" is written in Devanagari, Hinglish in Latin letters). Answers are capped at 512 tokens for chat and email and 384 for rewriting and translation, and a capped answer says that it was cut.

The worker's start. On this laptop Gemma-4-E2B decodes 12.1 tokens per second and reads a prompt at 58 tokens per second (E0, v1's measurement protocol), so long prompts dominate latency. Two changes make the on-demand worker cheap to start (Section 8.6): its weights are memory-mapped instead of copied, and the key–value state of its 1132-token few-shot prompt prefix is saved to a cache file after the first start and loaded afterwards, if and only if the saved token list equals the prefix.

Privacy. No component opens a network connection; the browser page binds to 127.0.0.1 by default and requires a key for its API. (A wifi-sharing option exists for a trusted local network over plain HTTP and is off.) Model-written code runs only inside the sandbox of Section 6.

4 The kind decision

Kinds. A request's kind is *app* (the old path), *chat*, *email*, *rewrite*, *translate*, *notes* (a question that the user's notes answer) or *code* (code to write, fix, shorten or test; explaining code is chat). The kinds were added in three dataset versions, Kinds-v0 to v2; Table 1 lists the two that were tested.

Head. One e5 pass embeds the request ("query:" prefix), and a multinomial logistic regression head gives a probability per kind. Its regularization C is chosen on validation macro-F1. Its temperature T is cross-fitted: train and calibration items are split into 5 folds by template family, each fold's logits come from a head trained on the other folds, and T minimises the negative log-likelihood of all out-of-fold logits; the final head is then trained on train and calibration together. (On Kinds-v0, a temperature fitted on its calibration split alone (84 to 103 items) made validation ECE worse in four of five development runs.)

Fallback and asking. If the head's top kind reaches τ, it decides: *app* goes the old way, a skill is written. Below τ, Gemma is prompted with the kinds' definitions and two examples per kind and language, decodes one label under a grammar that allows only the labels, and its confidence (the label's probability, temperature-scaled on the calibration split by binary cross-entropy, as v1's Tier 4) must reach τ_g for its kind to decide. Otherwise the head's top kind decides as before, except that a skill is not written but proposed to the user as a y/N question. Each decision is therefore one of four outcomes: *right*; *asked*; *sent the old way* (a skill request taken by the app path, which refuses it or takes it for another action); or *a skill not asked for* (a wrong skill written).

Choosing τ and τ_g. Both were chosen on the validation split, over $\{0.5, 0.6, 0.7, 0.8, 0.9\}^2$, by a rule fixed before the run: fewest wrong decisions (both wrong outcomes), then fewest questions, then the lower τ (fewer SLM calls), then the higher τ_g. For Kinds-v1 this gave $\tau = 0.7$, $\tau_g = 0.9$; for Kinds-v2, $\tau = 0.8$, $\tau_g = 0.7$ (142/152 right on validation). The service uses the Kinds-v2 head and settings.

5 Notes by meaning and notes Q&A

Retrieval. Each note is cut into chunks (its title and one paragraph), embedded once with the "passage:" prefix and cached by a hash of the note's text. The keyword search of v1 runs

Table 1: The v3 evaluation sets. All are synthetic: drafted with an AI assistant from families of English and Romanized Hindi (Hinglish) phrasings, and split by family. "Reviewed" means reviewed by the author; "claude reviewed" means the author waived the review and the AI assistant reviewed it, with a separate agent judging the files against written criteria. Kinds-v2's test split contains Kinds-v1's test split plus 24 fresh items.

Dataset	Labels / content	Items (train / cal / val / test)	Review
Kestrel-Kinds-v1	6 kinds	920 / 115 / 136 / 256	reviewed
Kestrel-Kinds-v2	7 kinds (+ code)	1000 / 123 / 152 / 280 (24 fresh)	claude reviewed
Kestrel-NotesQ-v0	18 fixture notes	dev 22 (4 none) / test 20 (2 none)	reviewed
Kestrel-NotesQA-v0	same notes	dev 26 (6 unans.) / test 25 (6 unans.)	reviewed
Kestrel-CodeTasks-v0	4 types	dev 24 / test 24 (24 families)	claude reviewed

first, unchanged; when it returns fewer than three notes, the single closest note by meaning (a note's score is its best chunk's cosine) is added if its score reaches θ, labelled "(by meaning)". A query that contains a common Hindi word is first put into English by Gemma. θ was chosen on development data to maximise the share of queries whose note was found minus the share of queries without a note that received one.

Q&A. A question of kind *notes* retrieves the $N = 2$ closest notes, none if even the closest is below θ. Gemma is told to answer only from them, in the question's language, or to say exactly `NOT IN NOTES`. A grounding check then requires every number and name in the answer to occur in the notes. The user sees a grounded answer with the note it came from, or the reply "The notes do not say this: so the reply is not available for this information" (the author's own wording), or, when the answer could not be checked, the closest note instead.

6 The code loop

A request of kind *code* goes to Gemma with a system prompt for short, standard-library Python (at most 640 tokens). The code is then checked in a sandbox: bubblewrap[1] with every namespace unshared (so no network), `/usr` read-only, an empty `/tmp` and a fresh work folder as the only writable place, a clean environment and no home directory; inside it, resource limits of 512 MiB of memory, 10 s of CPU, 10 MiB files and 64 processes, plus a wall-clock timeout, and `python3 -E -S`. Code is imported (so it compiles and its top level runs); code that waits for input at its top level is only compiled, since the sandbox has no input to give it. A syntax error, a crash or a timeout goes back to Gemma with the code, at most two times. The answer carries one line saying what was checked ("I checked it in a safe sandbox: no errors.") and a caveat. The evaluation harness (Section 8.4) checks by request type instead: a function or a fix is imported, a script is only compiled.

The loop deliberately does not run tests written by the model. In the first harness, models were asked to add their own checks and were retried when those failed; on development data Qwen3-4B's own checks failed on right code (wrong asserts, or asserts written where a sample input was expected) and caused 8 retries without changing a single outcome (Section 8.4).

7 Data and protocol

Datasets. Table 1 lists the sets. Kestrel-Kinds-v0 to v2 contain *app* items taken from the train, calibration and validation splits of v1's Kestrel-Intents-v0 (its test split was never read) plus families on the boundaries ("send an email" and "search my notes for the translation" are

[1] `https://github.com/containers/bubblewrap`

Table 2: Kind decisions on the test splits, each run once. "Skill req." are requests whose gold kind is a skill; "app req." are app requests. A taken decision is right, sent the old way, or a skill not asked for; the rest were asked.

	right				sent the	skill not
Policy (test split)	all	skill req.	app req.	asked	old way	asked for
Kinds-v1, head alone	206/256 (80.5%)	27/76	179/180	18	32	0
Kinds-v1, head + Gemma	220/256 (85.9%)	40/76	180/180	11	25	0
Kinds-v2, head alone	221/280 (78.9%)	40/96	181/184	37	20	2
Kinds-v2, head + Gemma	256/280 (91.4%)	78/96	178/184	13	5	6

Table 3: Right decisions per gold kind on the test splits.

	Kinds-v1 test		Kinds-v2 test	
Gold kind	head alone	head + Gemma	head alone	head + Gemma
app	179/180	180/180	181/184	178/184
chat	2/16	11/16	6/20	20/20
code	–	–	1/8	7/8
email	12/16	13/16	12/16	15/16
notes	1/12	1/12	7/16	12/16
rewrite	3/16	4/16	6/20	8/20
translate	9/16	11/16	8/16	16/16

app requests), and families for each skill. Kinds-v0 (5 kinds) was reviewed but never tested: its test was moved to Kinds-v1, which added *notes*. Kinds-v2 added 32 families: 12 code, 6 chat (among them explaining code and poems), 6 notes, 5 rewrite and 3 app; it keeps Kinds-v1's rows in their splits and adds 24 fresh test items. Kestrel-NotesQ-v0 holds paraphrase queries that share no keyword with their note, over 18 fixture notes (never the user's own), with queries that no note answers; Kestrel-NotesQA-v0 asks questions about the same notes, answerable ones with their key facts and unanswerable ones. Kestrel-CodeTasks-v0 has 24 families of four types (write a function, write a script, fix broken code, explain code), each with one English and one Hinglish request and hidden checks: asserts for functions and fixes, input/output cases for scripts (the last output line must contain the answer as whole words), and facts that a right explanation contains.

Protocol. Every choice (head, C, T, τ, τ_g, θ, prompts, model) was made on development splits. Each test split was then run once. Unlike v1, this protocol was not pre-registered as a whole: each step's rule was written down before its own run, sometimes after earlier steps' results were known (Section 10). Proportions are reported with 95% Wilson intervals (Wilson, 1927), computed from the logged counts. Timings come from the laptop as found (the memory runs record AC power, CPU boost on and Kestrel's own service idle), not from v1's controlled protocol, and are comparable only within one log.

8 Results

8.1 The kind decision

Table 2, Table 3 and Figure 2 give the test results. On Kinds-v1 the head alone took 206/256 decisions right; with the fallback, 220/256 (85.9%, CI $[81.1\%, 89.7\%]$), with no skill written that was not asked for. Every app request was right (180/180), but only 40/76 skill requests were (notes questions 1/12, rewrite requests 4/16), and 25 skill requests went the old way. Questions

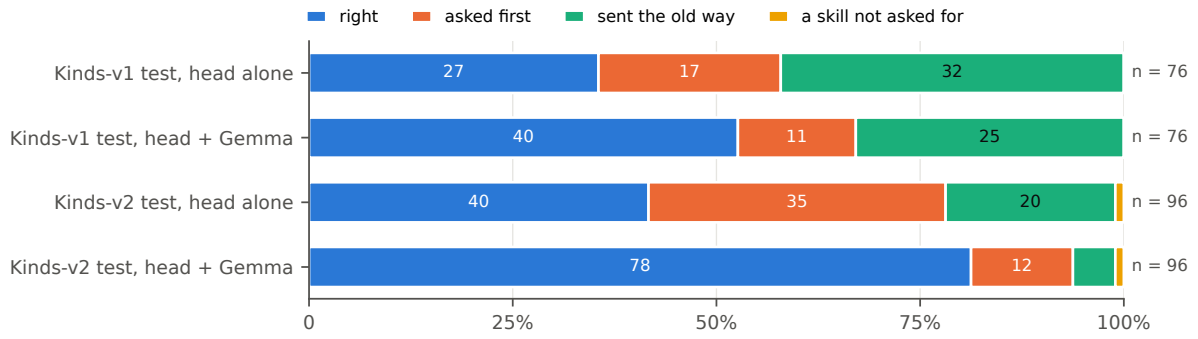


Figure 2: Decisions on requests for a skill (test splits). The fallback turns many questions to the user, and some requests sent the old way, into right answers. On Kinds-v1 a third of skill requests were still sent the old way; on Kinds-v2 most of those are gone, but 256 of its 280 test items had been scored once before (Section 10).

Table 4: Retrieval by meaning on Kestrel-NotesQ-v0. Every query's note is missed by the keyword search. "Right note added": the closest note is the query's note and reaches θ. The test split was run once, with θ from development.

Retrieval run	right note added	English	Hinglish	note added to a query with none
By meaning, no translation (dev, $\theta = 0.85$)	7/18	6/9	1/9	0/4
By meaning, Hinglish via English (dev, $\theta = 0.83$)	18/18	9/9	9/9	0/4
By meaning, Hinglish via English (**test**, $\theta = 0.83$)	14/18	7/9	7/9	0/2

about the user's own people and things ("which floor is the dentist on?") looked like app requests to both the head and Gemma.

On Kinds-v2, with the fallback, 256/280 decisions were right (91.4%, CI [87.6%, 94.2%]), 78/96 skill requests among them; 5 were sent the old way, 6 started a skill nobody asked for and 13 were asked. The fallback mattered most for chat and translation: the head alone was right for 6/20 chat and 8/16 translation requests, the head with Gemma for 20/20 and 16/16. It cost app requests a little (181/184 to 178/184): Gemma answered three jokes ("tell me a joke", "ek joke sunao", "kuch funny bolo") and "google exam preparation" as chat, which the data label app (small talk and web searches belong to the app). The head itself, above τ, sent "london mein abhi kitne baje hain?" to notes. Rewrite stayed weakest: 8/20 right, most of the others asked first.

The 24 fresh items give the cleanest view: 23/24 right with the fallback (CI [79.8%, 99.3%]) against 9/24 for the head alone, which asked 11 times. English and Hinglish were close (130/140 and 126/140).

The final heads' validation accuracy was 0.926 (Kinds-v1) and 0.862 (Kinds-v2), with macro-F1 0.799 and 0.761; temperature scaling lowered validation ECE from 0.147 to 0.076 and from 0.059 to 0.055. On test, the Kinds-v2 head reached accuracy 0.900, macro-F1 0.819 and ECE 0.072; its recall was lowest for chat (0.50) and code (0.63). On the validation items where the Kinds-v2 head's top kind was below τ, Gemma was right 30 times out of 38, the head 19 times. Gemma's label call took 523 ms at the median (p95 717 ms) on development data, once the model is loaded; one e5 pass and the head take a few milliseconds (v1 measured the encoder-alone system at a median of 5.6 ms per request).

8.2 Notes by meaning

No query's note was found by keywords (by construction), so every success in Table 4 is the meaning search's. Without translation, the right note was the closest one for 9/9 English but only 2/9 Hinglish development queries, and at the chosen $\theta = 0.85$ the search added it for 6/9 and

Table 5: Kestrel-CodeTasks-v0: requests right after the loop, by type (6 per type), and the mean generation seconds per request (model time; wall time is slightly longer). The first harness let models add their own tests; the final harness retries only on the sandbox's own errors. In every run, pass@1 equalled the result after the loop.

Model and run	function	script	fix	explain	all	gen. s / request
Gemma-4-E2B, first harness and checks (dev)	6/6	1/6	6/6	6/6	19/24	11.1
Qwen3-1.7B, first harness and checks (dev)	5/6	1/6	5/6	6/6	17/24	10.4
Qwen3-4B, first harness and checks (dev)	6/6	5/6	6/6	6/6	23/24	29.0
Gemma-4-E2B, final harness (dev)	6/6	5/6	6/6	6/6	23/24	9.1
Gemma-4-E2B, final harness (**test**, once)	6/6	3/6	6/6	6/6	21/24	8.9

1/9. Putting Hinglish queries into English first ("maa ke liye tohfa" became "gift for mother") made the right note the closest for all 18/18 development queries, and $\theta = 0.83$ added it for all of them without adding a note to any of the 4 queries that have none. On test the search found 14/18 (77.8%, CI $[54.8\%, 91.0\%]$), the same in both languages (7/9 and 7/9), and added no wrong note (0/2 on queries without a note). The closest note was right for 16/18; for two queries the right note was the closest but just below θ, and "festival shopping" ranked the Goa trip above the Diwali shopping note in both languages.

8.3 Notes Q&A

On the test split all 25/25 answers were graded right: 19/19 answerable questions answered with their facts and 6/6 unanswerable ones declined (CI $[86.7\%, 100.0\%]$ for the whole split). For every answerable question, the right note was among the two given. Generating an answer took 4.9 s on average (at most 6.8 s). On development data the same system declined only 4/6 unanswerable questions, both before and after the language note described below. Both misses asked for the price of a cake that the notes mention without a price: Gemma said in its own words that the notes do not state it, instead of the exact marker, so the app would have shown that sentence as an answer.

The answer's language lagged behind. The test run came before a note that tells Gemma to answer a Hinglish question in Hinglish; counted afterwards with the same word-list check, 4/10 answered Hinglish test questions got a Hinglish answer. With the note, 9/12 did on development data; the test split was not run again.

8.4 Code

With the first harness (Table 5), Qwen3-4B was right 23/24 times on development data, Gemma-4-E2B 19/24 and Qwen3-1.7B 17/24; all of Gemma's misses were scripts (1/6 right). A review of the hidden checks then made script grading fairer and fix checks stronger, and the same answers rescored to 22/24 for Gemma and 22/24 for Qwen3-4B (writing and fixing code: 16/18 vs 17/18), at 11.1 against 29.0 s of generation per request. Since Qwen3-4B would need a second model in memory and a second worker, Gemma was kept for code. Its explanations of Hinglish requests were Hinglish (3/3), against 0/3 for Qwen3-4B.

With the final harness Gemma was right 23/24 times on development data, and on the test split, run once, 21/24 times (87.5%, CI $[69.0\%, 95.7\%]$): all functions (6/6), fixes (6/6) and explanations (6/6), but only 3/6 scripts (Figure 3). The failed scripts read input until an uncaught end-of-file error, stopped at the first blank line, or reversed the line's letters when the Hinglish request ("words ulte order") asked for its words in reverse order. Generation took 8.9 s per request on average. The sandbox raised no error in any test answer, so the retry loop never ran (0 retries): its value here is as a check, not as a repair. Two weaknesses the scores do not show: explanations were asked for in the request's language, but all three English explanation requests

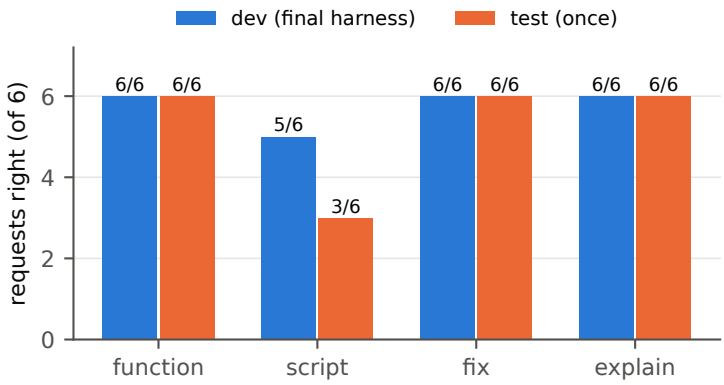


Figure 3: Gemma-4-E2B with the final harness on development data and on the test split (run once).

Table 6: Gemma-4-E2B (Q4_0) loaded with its weights copied or memory-mapped, page cache cold or warm (median of 3 runs each; one session). Output text was byte-identical in all runs.

Weights	page cache	load (ms)	first token (ms)	anonymous (MiB)	file-backed (MiB)
copied	cold	3656	111	3247	8
copied	warm	2596	103	3247	8
mapped	cold	2477	111	1144	2702
mapped	warm	1383	117	1144	2702

were answered in Hinglish (English: 0/3); and the fact-based grading passed an explanation of a list comprehension that stated a wrong output list.

8.5 Writing in the right language (development only)

On 23 development requests for rewriting and translation, a check of the answer's script and language passed 14/23 with the first prompts and 19/23 after the prompts named the output language and a note was added after the request. Translation into Hindi in Devanagari went from 0/4 to 4/4; translation into Hinglish fell from 3/4 to 1/4 (the English text came back unchanged). The prompts were changed while looking at these items, so these scores are optimistic.

8.6 System costs

Mapping the weights cut anonymous memory, which the kernel cannot reclaim, from 3247 to 1144 MiB (2103 MiB, 64.8%); the rest of the file, 2702 MiB, is file-backed and clean (Table 6, Figure 4). Peak-RSS counters still count those pages, so measured that way the mapped process looks larger. Loading became faster (cold 3656 to 2477 ms; warm 2596 to 1383 ms), and the first token came after 103–117 ms in every condition. The model's per-layer embeddings, almost half of the file, are still read at load: the installed llama.cpp binding has no setter for the lazy mode.

Loading is only part of a cold start. The service's worker primes a 1132-token few-shot prefix before it can answer. The v2 worker was ready after 19.1 s (median of 3); the v3 worker, computing and saving the prefix, after 17.4 s; and loading the saved prefix, after 2.0 s, 9.5 times faster than v2, with 1135 MiB of anonymous memory at ready against 3356 MiB.

8.7 Acceptance run

Before the version was frozen, every goal feature was driven end to end on a scratch service with the fixture notes. Over the socket, 34/36 requests passed (tasks with times in English and

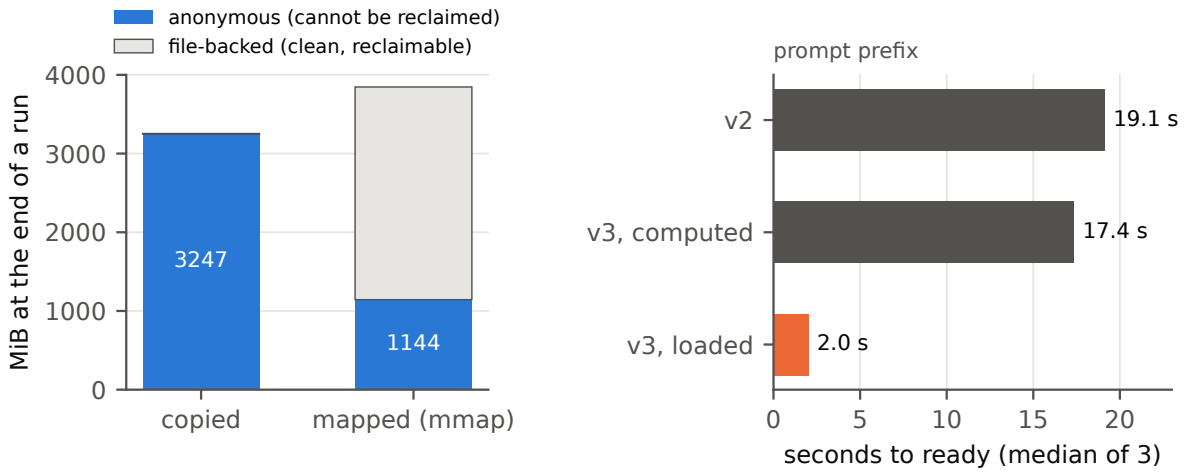


Figure 4: Left: memory at the end of a run, copied vs mapped weights. Right: time until the service's Gemma worker is ready, for the v2 worker and the v3 worker with its prompt prefix computed or loaded from the cache.

Hinglish, task commands, keyword and meaning search, summaries, memory, chat, email, rewrite, translation, notes Q&A, code, saving a note, refusals and the cut-off note); a first run had passed 31/35, with two failures caused by wrong checks in the script, which were corrected before a fresh rerun. The browser page passed 8/8 steps in headless Chrome and 8/8 in a visible window, including the Copy button read back from the clipboard. A reminder due at 17:31 was shown 2 s after its minute began. The automated suites passed (448 Rust tests, 510 Python tests, 6 sandbox tests and 4 model-worker tests).

The two failures are the limitations that matter most. "How much is my rent?" was refused: the head gave app 0.62, below τ, Gemma also chose app, and the question never reached notes Q&A, which answers it correctly when it does. "Which preferences have I set?" found nothing, because the memory search of v2 matches stored words. In this run 17/36 requests (47.2%) were answered in under a second: the greeting, creating and deleting a task, task commands, keyword and English meaning search, opening a document, memory, saving a note and a refusal (median 0.3 s); a translation took 2.8–3.2 s, an email 8.5–10.8 s, a notes answer 9.4 s and a written function 14.0 s (one run, not a benchmark: these times include whatever the worker was doing, such as starting).

9 Discussion

Ask, fall back, or refuse. The four-outcome view made the trade-offs visible. A higher τ turns wrong decisions into questions; the SLM fallback answers many of those questions itself (questions to the user fell from 37 to 13 on Kinds-v2, from 18 to 11 on Kinds-v1), at the cost of about half a second (Section 8.1) and occasional confident mistakes on the app's boundary (jokes, web searches). On Kinds-v1, sending a skill request the old way was by far the most common failure (25, against 0 skills not asked for); with Kinds-v2 the two were close (5 and 6). The old way is the costlier failure for the user, since the app path refuses the request or takes it for another action, and nothing tells them that a skill exists.

Data and settings changed together. Right decisions on skill requests rose from 40/76 on Kinds-v1 to 78/96 on Kinds-v2, but new families, a new Gemma prompt and new thresholds came together, and no run separates their effects. The rent question shows the limit of what synthetic families cover: the Q&A component answers it, but the decision never sends it there.

A 2B-class model is enough for small, well-specified code, not for I/O. Gemma matched the larger Qwen3-4B on our rescored development data in well under half of its time, and on test wrote every function and fix right. Scripts that read input are its weak spot, and the sandbox cannot catch them, because it has no input to give: it only proves that code compiles or imports.

Calibration still pays, unevenly. Cross-fitted temperatures lowered validation ECE for both heads, and the Kinds-v2 head's test ECE was 0.072 (validation: 0.055). But calibration cannot help where the data has no family: for the rent question both the head (below τ) and Gemma's fallback chose app.

10 Threats to Validity

Internal. Thresholds, prompts and the code model were chosen on development data, and each test split was run once. But the protocol grew step by step: Kinds-v2's new families were written after Kinds-v1's test errors were known, and 256 of Kinds-v2's 280 test items are Kinds-v1's test items, so the Kinds-v2 result on those items is optimistic; the 24 fresh items are the clean measure. The prompt changes for answer language were made while looking at development items. The code model was chosen after the checks were revised on the same development answers.

Construct. Explanations are judged by required facts, which a wrong statement can pass. Script grading reads the last output line. Notes Q&A grades were assigned by the evaluation script's rules and the author's review of the sets, not by a second annotator. Language is judged by a word list.

External. Every set is small (20 to 280 test items), synthetic, drafted with an AI assistant, and about one imagined user's life; two sets were reviewed by the assistant and an independent agent, not by the author. The notes are 18 fixture files. One laptop, one SLM, one encoder; timings were taken on the laptop as found.

Statistical. Intervals are wide: 21/24 has a 95% interval of $[69.0\%, 95.7\%]$, and 25/25 of $[86.7\%, 100.0\%]$. No hypothesis test was planned, and none is reported.

11 Limitations and Future Work

The acceptance run's README lists 12 known limitations, frozen with the version. Beyond the two failures above: requests to write a program "to solve X" go to chat (the program comes, unchecked); rewrite requests are often asked first; jokes go to chat and a time question to notes; Markdown is shown as typed; some notes answers to Hinglish questions are English; code uses the standard library only and is never run with input; the page can be shared on a wifi over plain HTTP when the user turns that on; and there is no memory of the conversation, so "continue" does not see the last answer. Energy was not measured.

The next round should start from the user's own misrouted requests: a Kinds-v3 and a CodeTasks-v1 built from them, reviewed by the author, with a fresh test split. A larger machine (v4) would allow larger models; on this laptop Qwen3-4B already decodes only 8.2 tokens per second.

12 Ethical Considerations

All inference runs on the user's machine, and the evaluation used fixture notes, never the user's own. Model-written code runs only in the sandbox, which has no network and no access to the user's files. Chat answers carry a note that an offline small model may be wrong, and notes answers name the note they came from. The page's wifi option exposes the notes to anyone on that network who has the link or can read its traffic; it is off by default and was off when this paper was written. The data sets and much of the code were drafted with an AI coding assistant and describe no real person.

13 Conclusion

A single 4-bit 2B-class model and a frozen encoder were enough to turn an intent classifier into an offline assistant that writes, translates, answers from the user's notes and writes small checked programs on an ordinary laptop. The decisive pieces were cheap: a calibrated head with a model fallback and a question to the user, translation before retrieval, an explicit "not in your notes", a sandbox that checks what it can, and two systems changes that make an always-on service start in about 2.0 s. The weak pieces are recognition of personal questions and rewrite requests, scripts that read input, and the language of some answers; they need real users' requests rather than more synthetic families.

Reproducibility. The system, its data sets, configurations and logs are in the repository at tag `v3-frozen`; this paper and its generator were added in a later commit on the same branch. Every measured value, share, interval and table entry in this paper is generated from the committed logs by `experiments/analysis/paper_assets_v3.py`; configuration constants (token caps, sandbox limits, N) and a few small counts in the prose (such as the number of development runs or of misrouted requests named) are typed and were checked by hand. `paper/v3/number_check.md` records independent re-derivations of the numbers and claims, with the corrections they led to.

References

Gaurav Arora, Shreya Jain, and Srujana Merugu. Intent detection in the age of LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1559–1570. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-industry.114. URL `https://aclanthology.org/2024.emnlp-industry.114/`.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL `https://arxiv.org/abs/2108.07732`.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024a. ISSN 2835-8856. URL `https://openreview.net/forum?id=cSimKw5p6R`.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant

Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, pages 8746–8825, 2024b. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html.

Arash Einolghozati, Abhinav Arora, Lorena Sainz-Maza Lecanda, Anuj Kumar, and Sonal Gupta. El Volumen Louder Por Favor: Code-switching in task-oriented semantic parsing. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 1009–1021. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.eacl-main.87. URL https://aclanthology.org/2021.eacl-main.87/.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Gemma Team. Gemma 4 technical report, 2026. URL https://arxiv.org/abs/2607.02770. Model used: gemma-4-E2B-it, model card at https://huggingface.co/google/gemma-4-E2B-it; GGUF weights from ggml-org/gemma-4-E2B-it-GGUF, revision b4243c156154b6dca9324415f8c7ccc098b4aed1; accessed 2026-10-24.

ggml-org. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2026. URL https://github.com/ggml-org/llama.cpp/commit/26394b4e6749a41c3633db040e0987500a5f7013. Commit 26394b4e6749a41c3633db040e0987500a5f7013 (2026-09-21); accessed 2026-10-24.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

Wittawat Jitkrittum, Neha Gupta, Aditya Krishna Menon, Harikrishna Narasimhan, Ankit Singh Rawat, and Sanjiv Kumar. When does confidence-based cascade deferral suffice? In *Advances in Neural Information Processing Systems*, volume 36, pages 9891–9906, 2023. doi: 10.52202/075280-0431. URL https://doi.org/10.52202/075280-0431.

Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1311–1316. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1131. URL https://aclanthology.org/D19-1131/.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=8sSqNntaMr.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-2124. URL https://aclanthology.org/P18-2124/.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3980–3990. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Neeraj Varshney and Chitta Baral. Model cascading: Towards jointly improving efficiency and accuracy of NLP systems. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11007–11021. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.756. URL https://aclanthology.org/2022.emnlp-main.756/.

Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings: A technical report, 2024. URL https://arxiv.org/abs/2402.05672.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953. URL https://doi.org/10.1080/01621459.1927.10502953.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.